Universidade Presbiteriana Mackenzie
Faculdade de Computação e Informática

Redes de computares

Prof. Dr. Bruno da Silva Rodrigues
Bruno.rodrigues@mackenzie.br

A comunicação não acontece corretamente.

No TCP, a conexão falha.
No UDP, a mensagem é enviada, mas o servidor não recebe.

Analise Programação de Socket UDP e TCP

Procedimento

- *Execute a IDLE do Python e abra os arquivos (Cliente e servidor) do socket UDP*
- *Cada arquivo deve ser aberto em uma IDLE diferente ou em computadores diferentes.*
- *Faça o mesmo procedimento para os arquivos socket TCP*

Após abrir e executar os arquivos, analise e responda:

Ian Merlini Kaltbeitzer 10402831

Pedro Esperança 10403152

Luan Ortiz 10403101

***Questão 1**. Execução dos programas (cliente e servidor) TCP e UDP:*

a) *Execute o cliente TCP antes de executar o servidor TCP. O que acontece? Por quê?*

O cliente tentar estabelecer a conexão com o servidor, não encontrará nenhum processo escutando nesse endereço IP e porta. Isso resultará em um erro de conexão e uma exceção será levantada no cliente indicando que a conexão falhou.

b) *Faça o mesmo procedimento para o cliente e servidor UDP. O resultado foi similar ao socket TCP? Compare os resultados e justifique.*

R:No UDP, o cliente consegue enviar dados mesmo sem o servidor ativo, pois o protocolo não estabelece conexão antes do envio. Já o TCP exige conexão prévia.

c) *O que acontece se o número da porta que o cliente tentar se conectar for diferente da porta disponibilizada pelo servidor?*

Questão 2. *Faça um chat entre cliente servidor (UDP ou TCP) onde ambos os lados trocam mensagens até uma das partes enviar o comando QUIT.* ***A porta do socket deve ser os primeiros 5 números do seu TIA.***

Servidor

```python
import socket

# Endereço IP e porta do servidor
SERVER_IP = '127.0.0.1'   # Use localhost se estiver rodando na mesma máquina
SERVER_PORT = 10402       # Primeiros 5 números do seu TIA

# Criação do socket TCP do servidor
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Associa o socket ao IP e porta
servidor.bind((SERVER_IP, SERVER_PORT))

# Coloca o servidor em modo de escuta
servidor.listen(1)

print("Servidor TCP esperando conexão na porta", SERVER_PORT)

# Aceita conexão do cliente
cliente, addr = servidor.accept()
print("Conexão estabelecida com", addr)

while True:
    # Recebe mensagem do cliente
    mensagem = cliente.recv(1024).decode()
    print("Cliente:", mensagem)

    # Verifica se o cliente quer sair
    if mensagem.upper() == "QUIT":
        break

    # Envia resposta ao cliente
    resposta = input("Você: ")
    cliente.sendall(resposta.encode())

    # Verifica se o servidor quer sair
    if resposta.upper() == "QUIT":
        break

cliente.close()
servidor.close()
```

Cliente

```python
import socket

# Endereço IP e porta do servidor
SERVER_IP = '127.0.0.1'
SERVER_PORT = 10402

# Criação do socket TCP do cliente
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM

# Conexão com o servidor
cliente.connect((SERVER_IP, SERVER_PORT))

print("Conectado ao servidor. Digite 'QUIT' para sair.")

while True:
    # Envia mensagem para o servidor
    mensagem = input("Você: ")
    cliente.sendall(mensagem.encode())

    # Verifica se o cliente quer sair
    if mensagem.upper() == "QUIT":
        break

    # Recebe resposta do servidor
    resposta = cliente.recv(1024).decode()
    print("Servidor:", resposta)

    # Caso o servidor também envie QUIT
    if resposta.upper() == "QUIT":
        break

cliente.close()
print("Conexão encerrada.")
```

Saídas

Servidor

```
-------- RESTART: C:\Users\cs431663\Downloads\La
Servidor TCP esperando conexão na porta 10402
Conexão estabelecida com ('127.0.0.1', 58705)
Cliente: teste
```

Cliente

```
C:\Users\cs431663\AppData\Local\Programs\Python\Python31
Conectado ao servidor. Digite 'QUIT' para sair.
Você: quit
Conexão encerrada.

Process finished with exit code 0
```

Objetivos da atividade:

Aprender a programar e Socket UDP e TCP assim como analisar e comparar o funcionamento de ambos protocolos da camada de transporte

Bibliografias

KUROSE, J. F. e ROSS, K. W. Redes de Computadores e a Internet – Uma Nova Abordagem – Pearson

Deitel, Harvey M.; Deitel, Paul J. - Java: como programar - 6ª edição - Pearson

Internet Engineering Task Force. Disponível em: https://www.ietf.org/t

Questão 3. *Faça uma aplicação qualquer usando o socket. Essa aplicação pode ser para enviar arquivos ou controlar algum objeto em uma das pontas da conexão ou gerenciar diversas conexões usado threads. Após elaboração do projeto, um vídeo deve ser gravado mostrando o funcionamento da aplicação e explicando o código.*

Código

Servidor:

Servidor.py - C:\Users\cs431663\Downloads\Lab redes questão 3\Servidor.py (3.12.6)

File Edit Format Run Options Window Help

```python
import socket
import threading

HOST = '127.0.0.1'
PORT = 10402

clientes = []
nomes = []


def broadcast(mensagem, cliente_remetente=None):
    clientes_remover = []

    for cliente in clientes:
        if cliente != cliente_remetente:
            try:
                cliente.sendall(mensagem.encode())
            except:
                clientes_remover.append(cliente)

    for cliente in clientes_remover:
        remover_cliente(cliente)


def remover_cliente(cliente):
    if cliente in clientes:
        indice = clientes.index(cliente)
        nome = nomes[indice]

        clientes.remove(cliente)
        nomes.remove(nome)

        try:
            cliente.close()
        except:
            pass

        print(f"{nome} desconectou.")
        broadcast(f"{nome} saiu do chat.")
```

```python
def tratar_cliente(cliente):
    try:
        nome = cliente.recv(1024).decode().strip()

        if not nome:
            cliente.close()
            return

        clientes.append(cliente)
        nomes.append(nome)

        print(f"{nome} entrou no chat.")
        cliente.sendall("Conectado ao chat com sucesso! Digite QUIT para sair.".encode())
        broadcast(f"{nome} entrou no chat.", cliente)

        while True:
            mensagem = cliente.recv(1024).decode().strip()

            if not mensagem:
                remover_cliente(cliente)
                break

            if mensagem.upper() == "QUIT":
                cliente.sendall("Você saiu do chat.".encode())
                remover_cliente(cliente)
                break

            texto_final = f"{nome}: {mensagem}"
            print(texto_final)
            broadcast(texto_final, cliente)

    except:
        remover_cliente(cliente)


def iniciar_servidor():
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    servidor.bind((HOST, PORT))
    servidor.listen()

    print(f"Servidor iniciado em {HOST}:{PORT}")
    print("Aguardando conexões...")

    while True:
        cliente, endereco = servidor.accept()
        print(f"Nova conexão de {endereco}")

        thread = threading.Thread(target=tratar_cliente, args=(cliente,))
        thread.start()


if __name__ == "__main__":
    iniciar_servidor()
```

Cliente

```python
import socket
import threading

HOST = '127.0.0.1'
PORT = 10402


def receber_mensagens(cliente):  1 usage
    while True:
        try:
            mensagem = cliente.recv(1024).decode()

            if not mensagem:
                print("Conexão encerrada pelo servidor.")
                break

            print(f"\n{mensagem}")
        except:
            print("\nConexão encerrada.")
            break

```

```python
def iniciar_cliente():  1 usage
    cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    cliente.connect((HOST, PORT))

    nome = input("Digite seu nome: ").strip()
    while not nome:
        nome = input("Digite um nome válido: ").strip()

    cliente.sendall(nome.encode())

    thread_receber = threading.Thread(target=receber_mensagens, args=(cliente,), daemon=
    thread_receber.start()

    while True:
        mensagem = input()

        cliente.sendall(mensagem.encode())

        if mensagem.upper() == "QUIT":
            break

    cliente.close()
    print("Cliente encerrado.")


if __name__ == "__main__":
    iniciar_cliente()
```

Saídas

Servidor

```
===== RESTART: C:\Users\cs431663\Downloads\Lab redes
Servidor iniciado em 127.0.0.1:10402
Aguardando conexões...
Nova conexão de ('127.0.0.1', 56494)
Ian entrou no chat.
Ian desconectou.
```

```
Digite seu nome: Ian

Conectado ao chat com sucesso! Digite QUIT para sair.
quit
Cliente encerrado.
Conexão encerrada.
```

Tratar arquivos

*Se a opção escolhida for manipular arquivos, você deverá fazer **um chat** e oferecer um menu para transferência de arquivos. O programa deverá dar ao usuário a opção de escolher qual arquivo será enviado e transmitir o arquivo via socket.*

****será necessário estudar manipulação de arquivos. Não serão aceitos projetos onde o arquivo a ser enviado já é predefinido no código.***

Tratar várias conexões

Neste desafio, o programa será capaz de tratar e responder diversas conexões usando threads

**será necessário estudar threads.*

Outras interações

*Você pode usar seu projeto ou conceitos de **Pygame** usado em semestres anteriores e controlar objetos usando um computador remoto através do socket.*

Não serão aceitos!!!

- Programas que façam uma simples transferência de arquivos

- Usar o Python para fazer o jogo da velha

Entregáveis

Vídeo 1:

- ***Questões 1 e 2 da atividade (CHAT simples) – 3,0 pontos***
  - Itens a serem verificados: correta explicação do cenário e do que está acontecendo na questão 1; demonstração clara do funcionamento da atividade da questão 2.

Vídeo 2:

- ***Questão 3 Desafio – 5,0 pontos***
  - Itens a serem verificados: originalidade e grau de dificuldade da aplicação, bem como demonstração clara do funcionamento da aplicação da questão 3.

Mesmo você não conseguindo fazer o exercício 3, você deverá apresentar a ideia para os colegas contando as dificuldades enfrentadas no desenvolvimento da atividade.

Cada vídeo pode ter no máximo 4 minutos de duração. O vídeo deve ser realizado na posição paisagem, ser disponibilizado no Youtube, conter o(s) nome(s) do(s) componente(s) do grupo, narração e demonstrar o funcionamento da atividade (vídeo 1) e da aplicação (vídeo 2).

Repositório no GitHub:

- ***Repositório com os códigos das atividades 2 e 3 – 2,0 pontos***
    - Itens a serem verificados: documentação clara dos códigos e a que eles se propõem; disponibilização dos códigos utilizados nas atividades 2 e 3; link dos vídeos de demonstração (vídeos 1 e 2).

Os arquivos de código não devem estar compactados em hipótese nenhuma!

O repositório será avaliado sob a seguinte perspectiva: se um terceiro consegue reproduzir o funcionamento da(s) aplicação(ões) a partir da consulta ao repositório.

Os links dos vídeos 1 e 2, bem como o link do repositório deverão ser enviados na atividade do Moodle para apresentação em sala de aula.